

Προηγμένες Web Εφαρμογές



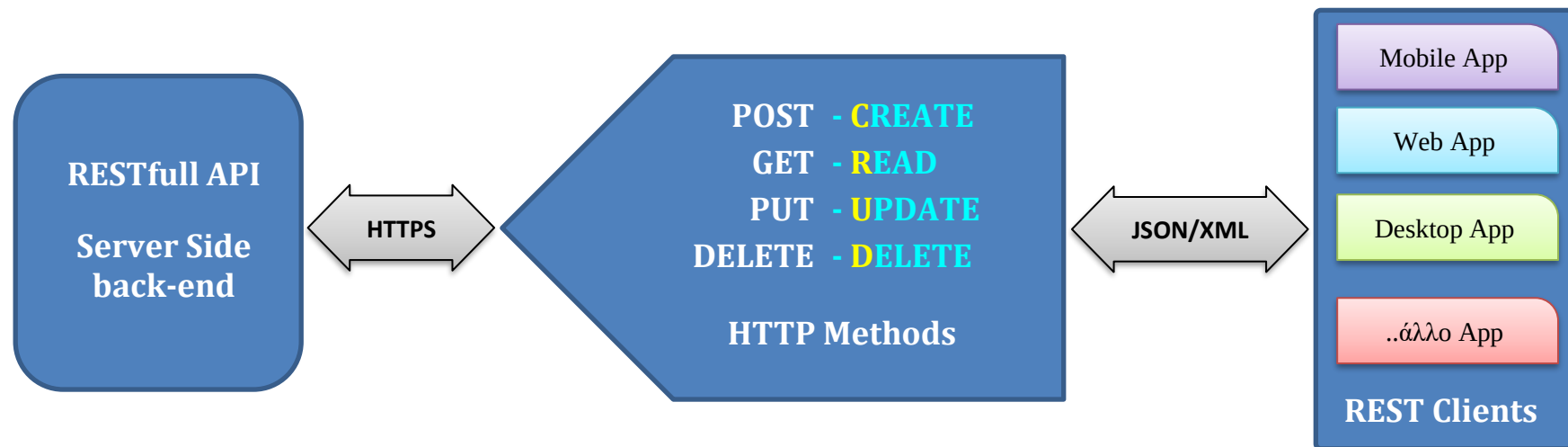
RESTfull APIs



Φώτης Κόκκορας
Πανεπιστήμιο Θεσσαλίας

Τι είναι ένα API / RESTfull API

- ❖ **API: Application Programming Interface** (Διεπαφή Προγραμματισμού Εφαρμογών)
 - ❑ για να χρησιμοποιήσουμε την λειτουργικότητα μιας εφαρμογής σε άλλη
 - ❑ δεν χρειάζεται να ξέρουμε πώς λειτουργεί η πρώτη εσωτερικά
- ❖ ΑΡΑ ένα API είναι μια διεπαφή (interface) για προγραμματιστές (και εφαρμογές).
 - ❑ Πρέπει να είναι λειτουργική.
 - ❑ Πρέπει να είναι "ευχάριστη" στη χρήση της (προβλέψιμη στη συμπεριφορά της).
- ❖ **REST: Representational State Transfer**
 - ❑ Αρχιτεκτονική Λογισμικού (Software Architecture)



- ❑ Επιτρέπει την εύκολη δημιουργία εφαρμογών με κοινό back-end και διαφορετικό front-end.



RESTful Web services

- ❖ Τι είναι: Web Υπηρεσίες (services) αρχιτεκτονικής REST
- ❖ Ένα RESTfull API είναι κατά βάση ένα web site με τη διαφορά ότι δεν απευθύνεται σε ανθρώπους για να το διαβάσουν (καταναλώσουν) αλλά σε άλλες εφαρμογές για να ανταλλάξουν με αυτό δεδομένα ή λειτουργικότητα.
 - ❑ Απευθύνεται σε άλλες web, desktop, mobile ή άλλες εφαρμογές με πρόσβαση στο web.
- ❖ Υπάρχουν πολλά RESTfull APIs
 - ❑ Portal για APIs: <https://www.programmableweb.com/>

Let's look at puppies.



hackett



hackett



hackett



hackett

...

/getAllDogs

/locationVerify

/foodNeeded

/createRecurringDogWalk

/giveDirectOrder

/healthCheck

/getRecurringDogWalkSchedule

/getLocation

/getDog

/massDogParty

/getNewDogsSince

/getRedDogs

/getSittingDogs

/dogStateChangesSearch

/replaceSittingDogsWithRunningDogs

/saveDog

...

A puppy's world is big.

```
...  
/getAllDogs  
/verifyLocation  
/feedNeeded  
/createRecurringWakeUp  
/giveDirectOrder  
/checkHealth  
/getRecurringWakeUpSchedule  
/getLocation  
/getDog  
/newDog  
/getNewDogsSince  
/getRedDogs  
/getSittingDogs  
/setDogStateTo  
/replaceSittingDogsWithRunningDogs  
/saveDog  
...
```

```
...  
/getAllLeashedDogs  
/verifyVeterinarianLocation  
/feedNeededFood  
/createRecurringMedication  
/doDirectOwnerDiscipline  
/doExpressCheckupWithVeterinarian  
/getRecurringFeedingSchedule  
/getHungerLevel  
/getSquirrelsChasingPuppies  
/newDogForOwner  
/getNewDogsAtKennelSince  
/getRedDogsWithoutSiblings  
/getSittingDogsAtPark  
/setLeashedDogStateTo  
/replaceParkSittingDogsWithRunningDogs  
/saveMommaDogsPuppies  
...
```


Keep the simple things simple.

We only need two base URLs per resource.

The first is for a collection.

/dogs

The second is for an element.

/dogs/**1234**

POST

GET

PUT

DELETE

CREATE

READ

UPDATE

DELETE

Resource	POST create	GET read	PUT update	DELETE delete
/dogs	create a new dog	list dogs	replace dogs with new dogs	delete all dogs
/dogs/1234	treat as a collection create new dog in it	show Bo	if exists update Bo if not create Bo	delete Bo

Resource	POST create	GET read	PUT update	DELETE delete
/dogs	create a new dog	list dogs	bulk update dogs	delete all dogs
/dogs/1234	error	show Bo	if exists update Bo if not error	delete Bo

Verbs are bad.

Nouns are good.

Plurals or singulars?

Foursquare

/checkins

GroupOn

/deals

Zappos

/Product

Plurals are better.

/dogs

Abstract or concrete naming?

Super High

/things

High

/animals

Medium

/dogs

Low

/beagles

Concrete is better than abstract.

/dogs

What about associations?

GET **/owners/5678/dogs**

POST **/owners/5678/dogs**

What about complex variations?



Cody Simms

Sweep variations under the ‘?’

`/dogs?color=red&state=running&location=park`

/dogs

What about errors?

Facebook

HTTP Status Code: 200

```
{"type": "OAuthException", "message": "(#803) Some  
of the aliases you requested do not exist:  
foo.bar"}
```

Twilio

HTTP Status Code: 401

```
{"status": 401, "message": "Authenticate", "code":  
20003, "more_info": "http://www.twilio.com/docs/  
errors/20003"}
```

SimpleGeo

HTTP Status Code: 401

```
{"code": 401, "message": "Authentication  
Required"}
```

Code for code

200 - OK

401 - Unauthorized

http://en.wikipedia.org/wiki/List_of_HTTP_status_codes

Message for people

```
{"message" : "Verbose, plain language  
description of the problem with hints about  
how to fix it."
```

```
"more_info" : "http://dev.teachdogrest.com/  
errors/12345"}
```

What about versioning?

Twilio

```
/2010-04-01/Accounts/
```

salesforce.com

```
/services/data/v20.0/subjects/Account
```

Facebook

```
?v=1.0
```

/v1/dogs

Please give me exactly what I need.

LinkedIn

```
/people:(id,first-name,last-name,industry)
```

Facebook

```
/joe.smith/friends?fields=id,name,picture
```

Google (partial response)

```
?fields=title,media:group(media:thumbnail)
```

```
/dogs?fields=name,color,location
```

What about pagination?

Facebook

```
offset  
limit
```

Twitter

```
page  
rpp
```

LinkedIn

```
start  
count
```

```
/dogs?limit=25&offset=50
```

What about formats?

Google Data

```
?alt=json
```

Foursquare

```
/venue.json
```

Digg*

```
Accept: application/json
```

```
?type=json
```

* The type argument, if present, overrides the Accept header.

`/dogs.json`

`/dogs/1234.json`

What about defaults?

Format

```
json
```

Pagination (depends on data size)

```
limit=10&offset=0
```

What about attribute names?

Twitter

 "created_at": "Thu Nov 03 05:19:38 +0000 2011"

Bing

 "DateTime": "2011-10-29T09:35:00Z"

Foursquare

 "createdAt": 1320296464

JSON is for JavaScript Objects

```
var myObject = JSON.parse(response);
```

These looks funny in JavaScript

```
timing = myObject.created_at;
```

```
timing = myObject.DateTime;
```

JavaScript Convention

```
"createdAt": 1320296464
```

```
timing = myObject.createdAt;
```

Medial Capitalization aka **CamelCase**

What about non-resource-y stuff?

Calculate

Translate

Convert

Use verbs **not** nouns

`/convert?from=EUR&to=CNY&amount=100`



What about searching?

Global 

```
/search?q=fluffy+fur
```

Scoped

/owners/5678/dogs/search?q=fluffy+fur

Formatted 

```
/search.xml?q=fluffy+fur
```

What about counts?

/dogs /**count**

What about the rest of the URL?

Facebook

graph.facebook.com

api.facebook.com

developers.facebook.com

Foursquare

api.foursquare.com

developers.foursquare.com

Twitter

api.twitter.com

search.twitter.com

stream.twitter.com

dev.twitter.com

API gateway

```
api.teachdogrest.com
```

Developer connection

```
developers.teachdogrest.com
```

Do web redirects

```
api → developers (if from browser)
```

```
dev → developers
```

```
developer → developers
```


What about exceptional stuff?

Client intercepts HTTP error codes

Twitter

```
/public_timelines.json?  
suppress_response_codes=true
```

```
HTTP Status Code: 200
```

```
{"error" : "Could not authenticate  
you." }
```

Always returns **OK**

```
/dogs?suppress_response_codes=true
```

Code for ~~code~~ ignoring

```
200 - OK
```

Message for people & code

```
{"response_code" : "401", "message" :  
  "Verbose, plain language description of the  
  problem with hints about how to fix it."  
  "more_info" : "http://dev.teachdogrest.com/  
errors/12345", "code" : 12345}
```

Client supports limited HTTP methods

Method Parameter

create

```
/dogs?method=post
```

read

```
/dogs
```

update

```
/dogs/1234?method=put&location=park
```

delete

```
/dogs/1234?method=delete
```

What about authentication?

PayPal

Permissions Service API

Facebook

OAuth 2.0

Twitter

OAuth 1.0a

Use latest and greatest OAuth

OAuth 2.0

Don't do something close, but different.

How do application developers use the API?

What about chatty applications?

First be complete & RESTful.

Then provide shortcuts.

Partial response syntax can help.

```
/owners/5678?fields=name,dogs(name)
```

Complement your API with code
libraries and SDK.

Really? All of this? And iterate it?

```
graph TD; Application[Application] --- AVL[API Virtualization Layer]; AVL --- API1[API]; AVL --- API2[API]; AVL --- API3[API];
```

Application

API Virtualization Layer

API

API

API

Be RESTful

Only 2 URLs

No verbs

Use nouns as plurals

Concrete over abstract

For JSON follow JavaScript conventions

Sweep complexity behind the ‘?’

Borrow from leading APIs

Account for exceptional clients

Add virtualization layer